

Deciding when a machine may give a customer their money back

An MCP server that resolves UPI Autopay debit complaints. When the bank's own records prove the debit should never have happened, it reverses the money in seconds. When they don't, it hands a human everything they need — and gets out of the way.

Project qna-action-mcp **Domain** Banking / UPI Autopay **Interface** Model Context Protocol **Status** Working end to end

01 The problem

A UPI Autopay mandate lets a merchant pull money from your account on a schedule without asking again. Most of the time that is a convenience. When it goes wrong it goes wrong in a specific, frustrating way: the money is already gone, and getting it back means convincing someone that it should not have left.

Today that conversation is a queue. The customer describes what happened, an advisor reads it, looks up the account, forms a judgement, and issues a credit or refuses one. The advisor is doing real work — but a large share of it is work a database could have done. "Was this debit above the limit the customer agreed to?" is not a matter of opinion. Neither is "did we send the pre-debit notice we are required to send?"

The naive fix is to have a language model read the complaint and decide. That fails badly, and it fails in the direction that costs money: the model is reading the account of the person asking to be paid. A confident, well-written complaint is not evidence. Neither is an emotional one.

02 The idea

Reverse money automatically **only** when the bank can prove the breach from its own records. Everything else goes to a human.

"Genuine" is not a mood read off the complaint text. A complaint is genuine, for the purposes of automatic reversal, when the mandate registry, the debit ledger and the pre-debit notice log — bank-side data the customer cannot influence — contradict the debit that was taken.

That single rule does most of the design work. It splits every complaint into two families, and the split is not about how serious the complaint is or how much money is at stake. It is about who holds the evidence.

03 Two families of complaint

Verifiable — the ledger settles it alone

No merchant needs to be asked, no analyst needs to form a view. The bank already holds a record that contradicts the debit. These reverse automatically.

REASON CODE	WHAT THE CHECK PROVES	VERDICT
MANDATE_REVOKED_BEFORE_DEBIT	The debit timestamp falls after the recorded revocation.	REVERSE
DEBIT_AFTER_MANDATE_EXPIRY	The debit falls outside the mandate's validity window.	REVERSE
AMOUNT_EXCEEDS_MANDATE_CAP	The amount is above the cap agreed at registration.	REVERSE
DUPLICATE_DEBIT	Same mandate, same amount, inside twenty-four hours.	REVERSE
MISSING_PRE_DEBIT_NOTIFICATION	No notice on record, or one sent under the required 24h.	REVERSE

Contested — someone outside the bank holds the evidence

These never reverse automatically, however sympathetic the complaint and however small the amount. Not because the customer is doubted, but because nothing in the bank's records can settle the question.

REASON CODE	WHY A PERSON DECIDES	VERDICT
UNAUTHORIZED_MANDATE	Suspected fraud. A false positive funds the fraudster, so an analyst signs off — but the mandate is paused in the same second.	HUMAN
SERVICE_NOT_RENDERED	Turns on what the merchant did or failed to do.	HUMAN
SUBSCRIPTION_CANCELLED_WITH_MERCHANT	Cancelled with the merchant, never with the bank. Our records show a live mandate, correctly.	HUMAN
AMOUNT_DISPUTED_WITH_MERCHANT	A pricing dispute. Needs merchant representment.	HUMAN

04 Guardrails: proof is necessary, not sufficient

A confirmed breach earns a reversal only if nothing else about the case argues for a person. Each of these pulls a verified claim back to a human — but sends it up with the finding and the amount already worked out, so the advisor's job is to approve, not to investigate from scratch.

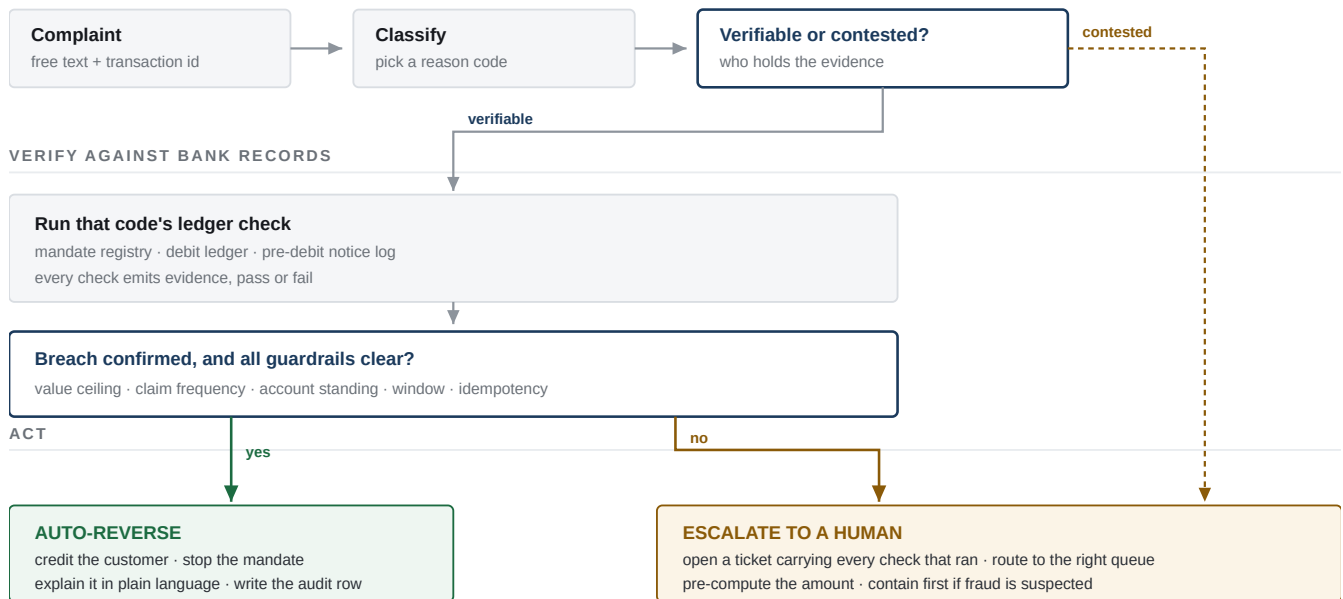
- **Value ceiling.** Above ₹25,000, a person authorises the credit.
- **Claim frequency.** Three auto-reversals already granted in ninety days routes the fourth to risk review.
- **Account standing.** Frozen accounts and unverified KYC take no automated credits.
- **Dispute window.** Debits older than ninety days need supervisory approval.
- **Idempotency.** A debit already reversed is never reversed again — an MCP host can retry a tool call on a timeout, and a retry that pays twice is a real loss.

WHY THERE IS NO AUTOMATIC REJECTION

A bank does not close a customer's money complaint without a person signing off. So when the ledger flatly contradicts the claim, the case still goes to a human — carrying a `LIKELY_INVALID` recommendation and the contradiction spelled out. The strongest thing the system does on its own authority is disagree in writing.

05 How a complaint moves through it

INTAKE



The load-bearing step is the second lane. Classification only chooses which ledger check to run; the check itself is what authorises money to move. The two outcomes are not "success" and "failure" — an escalation carrying five completed checks is a good outcome.

06 Why a crude classifier is safe here

Mapping free text to a reason code is done with keyword matching, which is unquestionably crude. It is safe anyway, and the reason is worth stating plainly, because it is the argument the whole design rests on.

Classification never decides that money moves. It decides *which verification to run*. Route a complaint to the wrong check and the check fails against the ledger, and the case escalates to a person. So the failure mode of a misread complaint is a slower answer — never a wrong payout. A test pins exactly this: force the engine to run the cap check on a debit that was within its cap, and it refuses to reverse.

This is also what makes it safe to let a language model participate. A model reading the complaint can propose a reason code and the engine will use it — and will still refuse to reverse if the ledger does not agree. The model gets to be helpful about *understanding*; it never gets to be authoritative about *paying*.

07 Shape of the system

The build is an MCP server, so any MCP host — Claude Desktop, Claude Code, a custom client — can hold the conversation while the server holds the authority. Tools come in three tiers, and the tiering is the security model: a model can call the read tools freely, should call the judgement tool before acting, and finds the effecting tools gated.

<code>reversal_policy.py</code>	The decision engine. A pure function of the case: no I/O, no clock reads, no model calls. Same inputs, same decision, forever — which is what makes the audit log worth anything, and lets 66 tests run in a seventh of a second.
<code>reversal_ledger.py</code>	The records the engine checks against — mandates, debits, pre-debit notices, accounts — plus the two writes a reversal needs, and an append-only audit log.
<code>reversal.py</code>	Carries a decision out: moves the money, stops the mandate, opens the ticket, drafts what the customer is told. Everything that can cause an effect lives here, and it is short enough to read in one sitting.
<code>mcp_server.py</code>	The MCP surface. <code>assess_reversal</code> runs the full engine and changes nothing, so a model can say what will happen before it happens. <code>initiate_refund_or_reversal</code> re-runs the engine internally and refuses if it does not independently reach the same verdict.
<code>core.py</code> · <code>configs/</code>	The existing domain-configurable layer — banking, government, healthcare, telecom — with per-domain intents, actions, knowledge and persona. The reversal engine plugs into the banking domain without disturbing the other three.

The split between the third and first entries is the one that matters. The hard part — deciding — is a pure function you can test exhaustively in milliseconds. The risky part — acting — is a short, boring file. Neither is allowed to become the other.

08 Two cases, end to end

TXN1001 · Netflix India · ₹649.00		AUTO-REVERSE
<i>"I cancelled the Netflix autopay mandate last week from my UPI app but they still took 649 rupees from my account on Monday. Please refund it."</i>		
CLASSIFIED	MANDATE_REVOKED_BEFORE_DEBIT	
CHECK	Revocation recorded 22 Aug 15:07. Debit executed 25 Aug 15:07 — three days after. Confirmed.	
GUARDRAILS	₹649 is well under the ceiling; first claim in ninety days; account in good standing; two days old. All clear.	
ACTION	₹649.00 credited back, reference <code>REV4197170619</code> . Mandate was already revoked on our side, so the failure was the merchant presenting against a dead mandate — flagged for merchant follow-up rather than pointlessly revoked twice.	
CUSTOMER HEARS	Why they were right, what was credited, when it lands, and what is being done about the merchant.	

"My InsureCo policy mandate had expired but they still collected 48,000 rupees from my account."

CLASSIFIED DEBIT_AFTER_MANDATE_EXPIRY

CHECK Mandate valid until 07 Aug. Debit executed 22 Aug. **Confirmed** — the customer is almost certainly owed this money.

GUARDRAILS ₹48,000 exceeds the ₹25,000 automatic ceiling. Stop.

ACTION Routed to **high-value-reversals** at high priority with recommendation **LIKELY_VALID**, the evidence attached and the reversal amount pre-computed. The advisor approves a finding; they do not repeat the investigation.

CUSTOMER HEARS That a person is looking at it, and by when — within four hours.

Nine scenarios like these ship with the project, one per branch the engine can take. Four escalate, and they escalate for four different reasons — which is the point. "Send it to a human" is not one behaviour. A fraud case gets the mandate stopped in the same second. A high-value case arrives with the arithmetic done. A contradicted claim arrives with the contradiction spelled out.

09 What this is honest about

The demo is real — no API key, no network, the actual decision path rather than a script — but a demo is not a deployment. The gaps worth naming:

- Reversals are written to a JSON store. Real money movement needs a transactional store and an idempotency key supplied by the caller, not just the transaction id.
- The ninety-day reversal counter increments but never ages back down. A real implementation counts reversal rows inside a rolling window.
- The notice check treats a missing row as proof no notice was sent. If the notification service can silently drop rows, that assumption pays out money it should not — this is the single most dangerous assumption in the system, and it is a data-integrity problem, not a logic one.
- Reason codes cover the common breaches, not every one. An unmapped complaint escalates, which is the correct failure, but it is still an unmapped complaint.

THE ONE-LINE VERSION

Most of the value here is not automation. It is drawing a defensible line between the complaints a machine can settle and the ones it must not — and then making the machine unable to cross it, even when a persuasive customer or a confident model would like it to.